

Article

Optimizing IoT Intrusion Detection —A Graph Neural Network Approach with Attribute-Based Graph Construction

Tien Ngo, Jiao Yin, Yong-Feng Ge and Hua Wang

Special Issue

Data Privacy Protection in the Internet of Things

Edited by

Dr. Xu Zheng, Dr. Zhuojun Duan and Prof. Dr. Yingjie Wang



Article

Optimizing IoT Intrusion Detection—A Graph Neural Network Approach with Attribute-Based Graph Construction

Tien Ngo , Jiao Yin , Yong-Feng Ge  and Hua Wang 

Institute for Sustainable Industries and Liveable Cities, Victoria University, Melbourne, VIC 3011, Australia; tien.ngo4@live.vu.edu.au (T.N.); yongfeng.ge@vu.edu.au (Y.-F.G.); hua.wang@vu.edu.au (H.W.)

* Correspondence: jiao.yin@vu.edu.au

Abstract: The inherent complexity and heterogeneity of the Internet of Things (IoT) ecosystem present significant challenges for developing effective intrusion detection systems. While graph deep-learning-based methods have shown promise in cybersecurity applications, existing approaches primarily construct graphs based on physical network connections, which may not effectively capture node representations. This paper proposes a Top-K Similarity Graph Framework (TKSGF) for IoT network intrusion detection. Instead of relying on physical links, the TKSGF constructs graphs based on Top-K attribute similarity, ensuring a more meaningful representation of node relationships. We employ GraphSAGE as the Graph Neural Network (GNN) model to effectively capture node representations while maintaining scalability. Furthermore, we conducted extensive experiments to analyze the impact of graph directionality (directed vs. undirected), different K values, and various GNN architectures and configurations on detection performance. Evaluations on binary and multi-class classification tasks using the NF-ToN IoT and NF-BoT IoT datasets from the Machine-Learning-Based Network Intrusion Detection System (NIDS) benchmark demonstrated that our proposed framework consistently outperformed traditional machine learning methods and existing graph-based approaches, achieving superior classification accuracy and robustness.

Keywords: network intrusion detection system; graph neural network; similarity-based graph construction; binary classification; multi-class classification



Academic Editor: Sokratis Katsikas

Received: 9 April 2025

Revised: 1 June 2025

Accepted: 9 June 2025

Published: 16 June 2025

Citation: Ngo, T.; Yin, J.; Ge, Y.-F.; Wang, H. Optimizing IoT Intrusion Detection—A Graph Neural Network Approach with Attribute-Based Graph Construction. *Information* **2025**, *16*, 499. <https://doi.org/10.3390/info16060499>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

1.1. The Rise of the IoT

For many years, the use of data collected from the Internet of Things (IoT) through uploading, sharing, and utilizing has significantly transformed how people work, study, and socialize. Given the rapid growth in IoT devices in recent years, it is clear that the IoT represents a promising paradigm to connect smart devices, providing users with seamless and flexible remote monitoring, improved control, and automation. This paradigm has contributed to enhancing productivity across multiple sectors, such as healthcare [1], agriculture [2], and transportation [3]. However, with the exponential growth in interconnected devices globally, the IoT has become a prime target for cyber attackers seeking to exploit its valuable information. The lack of robust security mechanisms in many IoT devices makes them particularly vulnerable to malicious attacks through security flaws, raising concerns about data security and privacy protection. The consequences of these attacks can range from data breaches and privacy violations, to significant losses and damage to critical applications if they remain undetected [4]. This underscores the need for implementing a

robust and intelligent IoT cybersecurity intrusion detection system to detect and mitigate attacks on IoT networks.

1.2. Network Intrusion Detection System

While traditional security mechanisms such as firewalls, antivirus software, and Intrusion Detection Systems (IDS) have been used for many years by companies and organizations, they fall short in addressing the unique characteristics of IoT systems. Due to their limited resources and the ability to store heterogeneous data from various sources, the design of IoT devices is more complex than other technologies, such as Radio Frequency (RF). This complexity, however, provides IoT devices with the flexibility to integrate with other technologies or systems. Yet, these characteristics raise questions about the feasibility of implementing a universal intrusion detection solution for IoT systems. A study by Li et al. (2021) highlighted that the sources of cyber threats can vary significantly, ranging from internal dissatisfaction or sabotage, to organized hackers and terrorists [4]. This further complicates the design of a Network Intrusion Detection System (NIDS), as hackers can also leverage Artificial Intelligence (AI) to launch sophisticated attacks aimed at stealing or disrupting data [5]. Effectively detecting such attacks and using them to train models requires substantial time and effort from researchers.

Although Machine Learning (ML)-based IDSs have been utilized in recent decades to address data scarcity and quality issues, these approaches often require large volumes of high-quality data for model training and validation, which is not always available for IoT datasets [6]. Research has shown that collecting fully labeled IoT network traffic datasets, which include a variety of attack scenarios, is challenging and time-consuming [7]. This aligns with the findings of Guerra et al. (2022) [7], who noted the scarcity of fully labeled network traffic datasets. Even with fully labeled datasets, the presence of imbalanced data makes it difficult to achieve effective generalization in ML-based IDSs [8]. As a result, the challenge of processing unbalanced or incompletely labeled datasets has recently received attention from both researchers and practitioners [9,10].

Convolutional Neural Networks (CNNs) are widely recognized for their efficiency in processing time and their strong capability to extract features from raw Euclidean-structured data. However, they are computationally intensive [11], and their performance tends to be highly dependent on the dataset used for training [12]. Other approaches, such as time series networks and Transformer-based models, have also shown promise in IDS development [13,14]. Nevertheless, these models come with their own limitations: time series networks often struggle with scalability [13], while Transformers are associated with high computational cost and limited interpretability [15].

While deep feature learning focuses solely on extracting and learning from individual data attributes, attribute-based Graph Neural Networks (GNNs) are capable of leveraging both node-level features and the structural relationships between entities. This aligns closely with the objectives of our proposed NIDS, which is designed to analyze network traffic by capturing both the characteristics of individual data points and their interactions within the network. In this context, along with sharing the same foundational principles as other deep learning (DL) models, GNNs offer a significant advantage over traditional deep feature mining approaches—as well as over conventional models such as CNNs, time series networks, and Transformers—by incorporating relational information into the learning process. Additionally, the ability to model IoT data as knowledge graphs enables efficient storage and semantic reasoning [16,17], further reinforcing the suitability of GNNs for intrusion detection tasks.

1.3. Graph Neural Networks

Temporal communication patterns, device-specific behaviors, and protocol diversity are key characteristics of IoT network flows and can significantly enhance the detection of anomalous events. By modeling these flows, graph-based learning approaches gain a distinct advantage over traditional methods, as they can more effectively capture the complex interactions and contextual relationships between devices. Leveraging these intrinsic properties is essential for designing NIDS solutions that are both scalable and effective in real-world IoT environments. Recent years have seen GNNs with great potential for addressing IoT challenges related to cyber threat detection. By learning node embeddings and capturing structural information from graph-structured data, GNNs can process complex relationships within network traffic, enabling them to effectively identify abnormal patterns and detect cyber threats that might evade traditional signature-based NIDSs [18]. This capability allows GNNs to offer a comprehensive representation of the system's state, while effectively addressing the class imbalance commonly found in IoT environments [19]. Moreover, GNNs can be combined with approaches such as federated learning [20] or graph anonymization [21] to ensure user privacy is maintained [22]. A number of GNN architectures have already been successfully applied across various NIDS domains, including the IoT, IIoT, and remote sensing. Additionally, their ability to operate in a decentralized manner [23] further strengthens their suitability for enhancing IoT security. However, significant work remains in areas such as network scalability, robustness to adversarial attacks, integration with existing security infrastructure, and improving explainability and interpretability [24] to fully realize the potential of graph-based systems in real-world applications [25].

There are various types of GNNs used in NIDS applications, each with their own strengths and limitations. The most commonly used is the Graph Convolutional Network (GCN), which employs convolution operations to aggregate node information equally from neighboring nodes, making it suitable for general graph tasks. On the other hand, a Graph Attention Network (GAT) introduces self-attention mechanisms that assign different weights to neighbors during aggregation. However, a GAT can face challenges when applied to large graphs, due to its complexity [26]. GATs have been applied in fields such as healthcare, bioinformatics, and social network analysis. The Graph Sample and Aggregation (GraphSAGE) model is the newest of these three. As its name suggests, GraphSAGE uses a sampling approach for neighbors, combined with multiple aggregation methods for generating node embeddings. This design allows GraphSAGE to scale more efficiently, making it better suited for large, dynamic IoT graphs [27] compared to GCNs and GATs.

1.4. Research Gap and Contributions

Traditional ML- and DL-based IoT NIDS algorithms often rely heavily on a dataset's feature selection phase, which can be time-consuming and dependent on understanding the dataset from a machine perspective. While graph-based methods are advantageous for this problem, existing works that rely on physical network connections for graph construction have significant limitations. These methods may not effectively capture the authentic relationships between nodes, leading to suboptimal performance. To address this, we propose a novel Top-K Similarity Graph Framework (TKSGF) for IoT network intrusion detection.

The main contributions of this paper are as follows:

- We introduce TKSGF for IoT network intrusion detection. Our framework achieved superior performance on both binary and multi-class classification tasks using the

NF-ToN IoT and NF-BoT IoT datasets, outperforming traditional machine learning methods and existing graph-based approaches.

- We develop a novel Top-K Attribute Similarity-Based Graph Construction method, rather than the existing physical-network-connection-based method. Our method ensures a more meaningful representation of node relationships and achieved better results compared to physical-link-based methods.
- We conducted comprehensive experiments to investigate the impact of graph directionality (directed vs. undirected), different K values, and various GNN architectures and configurations on IoT NIDS detection performance. These experiments provided valuable insights and reference points for future research and practical applications in the field.

This paper is divided into five sections. Section 2 describes the related works. Section 3 focuses on the study's materials and methods. Section 4 contains the presentation and discussion of results, along with suggestions for further study. Finally, Section 5 presents the conclusions.

2. Related Work

2.1. Machine Learning in IoT Cyber Security

Machine learning is a fundamental technique that provides quick and effective solutions to common IoT challenges, such as intrusion detection and anomaly detection. Previous studies have demonstrated that with proper hyperparameter tuning, ML can deliver competitive results in monitoring suspicious events, identifying abnormalities, and detecting malicious activities, thereby contributing to the overall enhancement of IoT security. Moreover, ML plays a crucial role in creating benchmark datasets that can be used to evaluate the performance of IoT security systems using state-of-the-art (SOTA) methods. These benchmark datasets can also be applied alongside deep learning techniques for performance comparisons. In IoT cybersecurity environments, various ML models can be utilized, including Decision Trees (DT), Random Forest (RF), K-Nearest Neighbors (KNN), Naive Bayes (NB), and Extreme Gradient Boosting (XGBoost).

Previous studies have highlighted the significant role of ML in advancing cybersecurity. For instance, Guezzaz et al. (2021) developed a decision tree classifier and enhanced data quality, achieving robust accuracy rates of 99.42% and 98.80% with the NSL-KDD and CICIDS2017 datasets, respectively [28]. Majidian et al. (2024) focused on optimizing the random forest classifier by partitioning the network into controller nodes and grouping them into subdomains, yielding average accuracies of 98.06% and 99.67% on the NSWNB15 and NSLKDD datasets, respectively [29]. Recognizing the importance of data quality for improving detection capabilities, Mohy et al. (2023) [30] proposed three feature selection methods—principal component analysis (PCA), univariate statistical tests, and genetic algorithms (GA)—to select a subset of features that effectively represent the entire dataset. They demonstrated the impact of these methods using K-NN, which resulted in outstanding performance and a significant reduction in prediction times [30]. Mehmood et al. (2018) explored ways to improve the detection rate of Distributed Denial-of-Service (DDoS) attacks by implementing a Naïve Bayes classification algorithm with multiple agents, reporting very fast performance [31]. Alqahtani et al. (2020) applied a genetic-based extreme gradient boosting (GXGBoost) algorithm to a Fisher-score-based feature selection method, significantly reducing the number of required data traffic features, while achieving a high IoT botnet attack detection rate [32]. Stacking different ML models can also enhance overall performance. For example, Douiba et al. (2023) presented an improved IDS by combining gradient boosting (GB) and decision trees (DT), achieving perfect precision and recall for malicious classes such as Password, SQL injection, Uploading, and Vulnerability Scanner

on the Edge-IIoT dataset [33]. Despite challenges such as data availability and quality, vulnerability to adversarial attacks, resource constraints, and privacy concerns, ongoing research continues to seek better solutions to enhance IoT security and the resilience of systems.

2.2. Deep Learning in IoT Cyber Security

Most IoT datasets consist of network traffic records, which typically require manual feature engineering in ML to extract relevant data features. In contrast, DL leverages its automatic feature extraction capability to learn complex patterns from raw data, enabling it to achieve competitive results in tasks such as attack detection and device-type identification. Hamidouche et al. (2023) demonstrated that DL can outperform ML in IoT cybersecurity tasks [34]. The study also suggested that DL models can transform unstructured network traffic data into images, showcasing their adaptability to classification problems. DL can be categorized into supervised learning, unsupervised learning, and transfer learning [35], where knowledge from one domain is applied to enhance learning in another.

In supervised learning, Halbouni et al. (2022) combined Convolutional Neural Network (CNN) and Long Short-Term Memory (LSTM) layers to extract both spatial and temporal features, achieving accuracies of 99.64%, 94.53%, and 99.67% for binary classification on the CIC-IDS2017, UNSW-NB, and WSN-DS datasets, respectively [36]. Ullah et al. (2022) introduced a hybrid model by placing recurrent layers after the convolution layer to promote more balanced learning [37]. Their model demonstrated consistent performance in both multi-class and binary classification tasks across multiple datasets, including NSLKDD, BoT-IoT, IoT-NI, IoT-23, MQTT, MQTTset, and IoT-DS2.

In the realm of unsupervised learning, Park et al. (2022) focused on integrating autoencoders with reconstruction error and Wasserstein distance-based generative adversarial networks to tackle the issue of data imbalance [38]. Meanwhile, Alrayes et al. (2024) [39] proposed using a denoising autoencoder for unsupervised learning and feature extraction to detect and prevent intrusion attempts in real time. Their approach achieved impressive F1-scores of 99.8% and 98.9% when tested on the CICIDS 2017 and NSL-KDD datasets, respectively [39].

2.3. Graph Deep Learning in IoT Cyber Security

Graph Deep Learning (GDL), which utilizes GNNs, is a branch of DL focused on learning from graph-structured data, such as modeling network traffic as a graph. This enables GNNs to develop flexible and scalable architectures that can effectively address cyber threats targeting IoT devices and networks. By learning node embeddings and capturing structural patterns in network traffic, GNNs can efficiently identify abnormal behaviors and detect cyber attacks that may elude traditional signature-based NIDSs. Additionally, GNNs offer a comprehensive representation of the system's state, while effectively handling the class imbalance commonly seen in IoT environments.

Several notable works have successfully integrated GDL into NIDSs using supervised learning. For example, Deng et al. (2022) introduced a novel Flow Topology-based Graph Convolutional Network (FT-GCN) approach that emphasizes traffic flow patterns to address label-limited IoT network intrusion detection [40]. Zhang et al. (2023) combined an enhanced version of a Graph Attention Network (GAT) with a Long Short-Term Memory (LSTM) network to effectively capture both the spatial topology and temporal features of network traffic, showing improvements in both binary and multi-class classification tasks [41]. Hamilton et al. (2017) laid the foundation for GraphSAGE, an inductive framework that utilizes node feature information to efficiently generate node embeddings [42]. Lo et al. (2022) [43] introduced the first E-GraphSAGE, a GNN approach that incorporates

both edge features and topological information for IoT network intrusion detection. Their extensive evaluation on four NIDS benchmark datasets highlighted the effectiveness of their model in terms of key classification metrics [43]. Lastly, Xu et al. (2024) presented the first GNN-based method for multi-class classification tasks in NIDSs using an unsupervised approach [44]. Their method efficiently distinguished normal network flows from malicious ones across different attack types, demonstrating strong generalization capabilities and potential for real-world traffic detection applications.

3. Methodology

3.1. Graph Theory Preliminaries

A graph is a mathematical structure used to model pairwise relations between entities. Formally, a graph is defined as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = v_1, v_2, \dots, v_N$ is the set of vertices (or nodes), and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges connecting pairs of nodes.

An alternative representation of a graph is the adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$, where each element A_{ij} denotes the presence or absence of an edge between node v_i and node v_j . In this study, edges are not determined by physical network topology, but instead by the attribute similarity between nodes, represented by a similarity matrix $\mathbf{S} \in \mathbb{R}^{N \times N}$. Each entry S_{ij} quantifies the similarity between nodes v_i and v_j . Given a top-K similarity $K \in \mathbb{N}$, the construction method of the adjacency matrix $\mathbf{A}$ will be detailed in Section 3.4.

3.2. The Overall Framework

The proposed Top-K Similarity Graph Framework for IoT network intrusion detection, unlike traditional methods that rely on physical network connectivity for graph construction, leverages feature-based similarity to construct more meaningful graph structures. As shown in Figure 1, the framework consists of four main stages: (1) data preprocessing and optional feature extraction, (2) graph construction based on Top-K attribute similarity, (3) node representation learning via Graph Neural Networks, and (4) classification for binary and multi-class intrusion detection tasks.

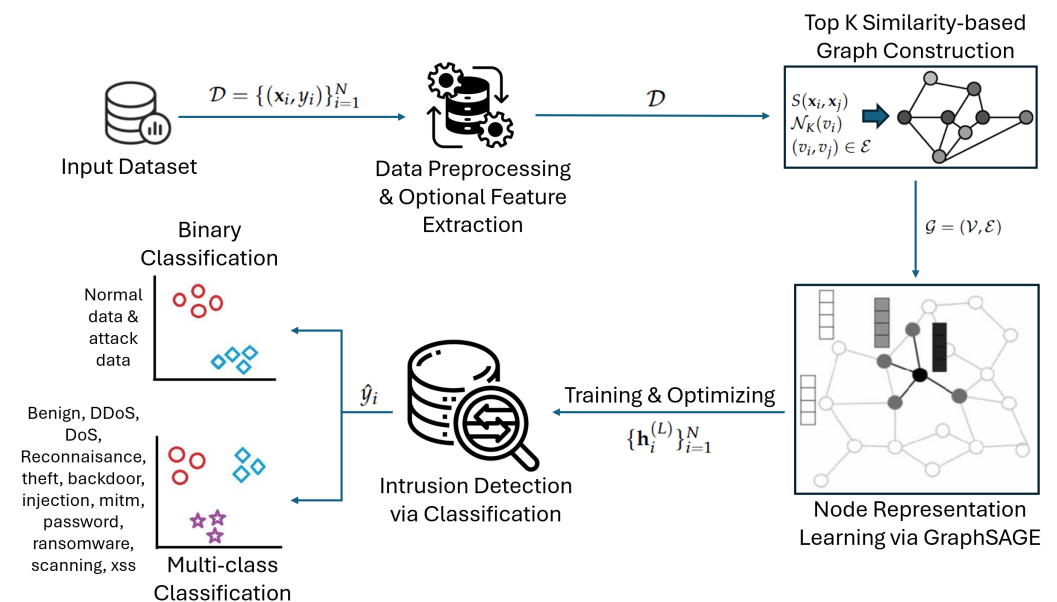


Figure 1. Proposed novel top-K similarity graph framework for IoT network intrusion detection.

3.3. Data Preprocessing and Feature Representation

Let the input dataset be represented as $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, where $\mathbf{x}_i \in \mathbb{R}^d$ denotes the d -dimensional feature vector of the i -th network flow and y_i is the corresponding class

label. Prior to graph construction, each $\mathbf{x}_i$ undergoes standard preprocessing procedures, such as normalization or encoding, to ensure uniformity in feature space.

3.4. Top-K Attribute-Similarity-Based Graph Construction

A graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is constructed where each node $v_i \in \mathcal{V}$ corresponds to a feature vector $\mathbf{x}_i$. Unlike traditional approaches, we construct edges based on attribute similarity rather than physical topology. To measure the similarity between two nodes, we use cosine similarity, as shown in Equation (1):

$$S(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{x}_i \cdot \mathbf{x}_j}{\|\mathbf{x}_i\| \|\mathbf{x}_j\|} \quad (1)$$

For each node v_i , we identify its Top-K most similar neighbors, as shown in Equation (2):

$$\mathcal{N}_K(v_i) = \text{Top-K}(\{S(\mathbf{x}_i, \mathbf{x}_j)\}_{j \neq i}) \quad (2)$$

Edges are established between v_i and each $v_j \in \mathcal{N}_K(v_i)$, as shown in Equation (3):

$$(v_i, v_j) \in \mathcal{E} \quad \text{if } v_j \in \mathcal{N}_K(v_i) \quad (3)$$

After determining the Top-K neighbors for each node v_i based on cosine similarity, we establish edges between nodes as described in Equation (3). At this stage, the graph can be constructed as either directed or undirected, depending on the desired relationship between the nodes.

Undirected Graph: In an undirected graph, the connection between nodes is mutual, meaning that if node v_i is connected to node v_j , then node v_j is also connected to node v_i . Therefore, the adjacency matrix $\mathbf{A}$ for the undirected graph is defined as Equation (4):

$$\mathbf{A}_{ij} = \begin{cases} 1, & \text{if } v_j \in \mathcal{N}_K(v_i) \text{ and } i < j \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

Here, an edge is established between v_i and v_j if their cosine similarity exceeds a predefined threshold $s_{\text{threshold}}$, with the condition $i < j$ ensuring that the matrix is symmetric, reflecting the bidirectional nature of the relationship.

Directed Graph: In a directed graph, edges have a direction, meaning that if node v_i is connected to node v_j , this does not imply that v_j is connected to v_i . The adjacency matrix $\mathbf{A}$ for a directed graph is defined as Equation (5):

$$\mathbf{A}_{ij} = \begin{cases} 1, & \text{if } v_j \in \mathcal{N}_K(v_i) \text{ and } i \neq j \\ 0, & \text{otherwise.} \end{cases} \quad (5)$$

In this case, an edge is created from v_i to v_j if the cosine similarity between their feature vectors exceeds the threshold. The condition $i \neq j$ ensures that there are no self-loops (i.e., a node is not connected to itself).

The choice between using a directed or undirected graph depends on the characteristics of the data and the specific task at hand. If the feature relationships are symmetric or bidirectional (e.g., in clustering tasks where the similarity between nodes should be mutual), an undirected graph is typically used. However, if the relationships are asymmetric (e.g., where one feature may be more influential or dominant in determining connections), a directed graph would better reflect these interactions.

This Top-K similarity-based construction allows the graph to represent more meaningful relationships between instances in the IoT dataset, accommodating both directed and undirected configurations for comparative experimentation.

3.5. Node Representation Learning with GraphSAGE

To capture contextual relationships between network flows, we apply GraphSAGE for inductive node embedding. In each layer l , the embedding of node v_i is updated by aggregating the embeddings of its neighbors, as shown in Equation (6):

$$\mathbf{h}_i^{(l)} = \sigma\left(\mathbf{W}^{(l)} \cdot \text{AGG}^{(l)}\left(\left\{\mathbf{h}_i^{(l-1)}\right\} \cup \left\{\mathbf{h}_j^{(l-1)} : j \in \mathcal{N}(v_i)\right\}\right)\right) \quad (6)$$

where $\mathbf{h}_i^{(l)}$ is the embedding of node v_i in layer l ; $\mathbf{W}^{(l)}$ is the learnable weight matrix; $\sigma(\cdot)$ is an activation function (e.g., ReLU); $\text{AGG}^{(l)}(\cdot)$ is an aggregation function such as mean, LSTM, or max-pooling; and $\mathbf{h}_i^{(0)} = \mathbf{x}_i$ is the initial feature vector. After L layers, the final node representation is denoted as Equation (7):

$$\mathbf{z}_i = \mathbf{h}_i^{(L)} \quad (7)$$

3.6. Intrusion Detection via Classification

The learned embeddings are used for intrusion classification. We apply a softmax classifier to predict the class label of each node:

$$\hat{y}_i = \arg \max_{c \in \{1, \dots, C\}} \text{softmax}(\mathbf{W}_c^\top \mathbf{z}_i + b_c) \quad (8)$$

where C is the number of classes, and $\mathbf{W}_c, b_c$ are the weights and bias for class c .

The model is trained using the cross-entropy loss function, as in Equation (9):

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log \hat{p}_{i,c} \quad (9)$$

where $y_{i,c}$ is a binary indicator (1 if sample i belongs to class c , 0 otherwise), and $\hat{p}_{i,c}$ is the predicted probability for class c computed by the softmax function.

This approach enables scalable and flexible intrusion detection by learning high-quality node embeddings that capture structural and semantic relationships among network flows.

4. Results and Discussion

4.1. Datasets

IoT datasets are generally categorized into real-world, synthetic, or hybrid types, depending on the context in which they are generated. Real-world datasets offer valuable insights into actual attack behaviors and system vulnerabilities, but they are often expensive and time-consuming to collect. In contrast, synthetic datasets are easier to generate but may lack the complexity and variability of real-world scenarios, potentially limiting their generalizability. Hybrid datasets aim to balance these trade-offs by combining real and synthetic data, resulting in more comprehensive and representative datasets that are widely adopted in research.

Developing an effective GNN-based algorithm relies on access to a reliable and comprehensive dataset for training, evaluation, and testing. In the context of NIDSs, the NetFlow format offers valuable IP flow data by capturing detailed network traffic patterns. This structured format not only facilitates the reconstruction of raw graph-based network structures, but also aids in identifying anomalous behaviors. As a result, NetFlow (NF) datasets

enhance the ability of researchers to visualize and analyze traffic flows more effectively, thereby supporting more informed experimentation with GNN models [45].

Most NF datasets used in IoT and Industrial IoT (IIoT) environments are hybrid in nature, combining real-world network traffic with simulated attack scenarios to create comprehensive benchmark datasets for NIDSs. These datasets often include various forms of information such as network traffic details, device health indicators, and system logs. Prior research has utilized a range of NF-based IoT datasets for intrusion and anomaly detection tasks, including NF-CSE-CIC-IDS2018, NF-BoT-IoT, NF-ToN-IoT, and updated versions such as NF-CSE-CIC-IDS2018-v2 and NF-BoT-IoT-v2.

In this study, NF-BoT-IoT and NF-ToN-IoT were selected due to their relevance in representing IoT network traffic and their demonstrated reliability and consistent performance in prior research. Both datasets are categorized as medium-to-large-scale, making them well suited for GPU-accelerated experimentation to enhance processing efficiency.

These datasets consist of both binary attack labels and detailed attack type annotations, enabling the evaluation of two distinct classification tasks: (1) binary classification, which determines whether a given network traffic sample is benign or malicious, and (2) multi-class classification, which identifies the specific category of malicious activity. The attack categories represented in the datasets are outlined below:

- Denial of Service (DoS): Disrupts the normal functioning of a system by overwhelming it with malicious requests or operations.
- Distributed Denial of Service (DDoS): A coordinated DoS attack executed from multiple sources to flood and incapacitate a target system.
- Reconnaissance: Involves probing or scanning activities aimed at gathering information about a system's vulnerabilities prior to launching an attack.
- Theft: Refers to unauthorized access and extraction of sensitive or personal information, often for illicit distribution or sale.
- Backdoor: Utilizes hidden entry points to gain unauthorized access to a system, bypassing standard authentication mechanisms.
- Man-in-the-Middle (MITM): An attack in which the adversary intercepts and potentially alters communications between two parties without their knowledge.
- Password Attack: Attempts to compromise user credentials in order to gain unauthorized access to protected systems or accounts.
- Ransomware: Malicious software that encrypts or locks system files, demanding payment from users in exchange for restored access. A notable example is the WannaCry ransomware worm, which propagated globally in 2017, encrypting files on affected systems and demanding payment in Bitcoin for decryption [46]. More information is available at <https://attack.mitre.org/software/S0366/>, accessed on 8 June 2025.
- Scanning: Involves automated probing to identify open ports, running services, or system vulnerabilities that may be exploited.
- Cross-Site Scripting (XSS): Injects malicious scripts into trusted websites or applications to exploit users and compromise system integrity.

4.2. Setup

The experiments were conducted on a single NVIDIA GeForce RTX 3070 GPU with 8GB of memory. PyCharm (2024.1.4 Professional Edition) was used as the primary integrated development environment (IDE), due to its robust support for Python programming, flexibility in coding, and powerful debugging tools. Its compatibility with a wide range of libraries also contributed to its selection. These include NumPy (1.26.4) for numerical operations, Pandas (2.2.3) for data manipulation, FAISS (1.10.0) for efficient similarity search and vector clustering, scikit-learn (1.6.1) for machine learning algorithms, PyTorch (2.1.0+cu118)

for building deep neural networks, and Matplotlib (3.10.1) and Seaborn (0.13.2) for data visualization.

The public datasets utilized in this paper are available for access and download from the Machine-Learning-Based NIDS websites (https://staff.itee.uq.edu.au/marius/NIDS_datasets/, accessed on 11 June 2025). A minimum of 60% of each dataset was allocated to the training set, with the remaining data evenly split between the validation and test datasets. Following the approach of Altaf et al. (2023), we adopted various train-to-test ratios [47], selecting 70% of the NF-BoT-IoT data and 60% of the NF-ToN-IoT data for training. The attack distribution of the tested datasets is summarized in Table 1.

Table 1. Attack distribution in NF-BoT IoT and NF-ToN IoT datasets.

Dataset	Label	Attack	Quantities	Train	Eval	Test	Total Data
NF-BoT IoT	0	Benign	13,859	9701	2079	2079	600,100
	1	DDoS	56,844	39,791	8527	8526	
		DoS	56,833	39,783	8525	8525	
		Reconnaissance	470,655	329,459	70,598	70,598	
		Theft	1909	1336	286	287	
NF-ToN IoT	0	Benign	270,279	162,167	54,056	54,056	1,379,274
	1	Backdoor	17,247	10,348	3449	3450	
		DDoS	326,345	195,807	65,269	65,269	
		DoS	17,717	10,630	3543	3544	
		Injection	468,539	281,123	93,708	93,708	
		MITM	1295	777	259	259	
		Password	156,299	93,779	31,260	31,260	
		Ransomware	142	85	28	29	
		Scanning	21,467	12,880	4293	4294	
		XSS	99,944	59,966	19,989	19,989	

For each dataset, categorical labels were numerically encoded into features, while the labels were transformed into binary and multi-class formats as appropriate. Depending on the programming language used, both types of labels were converted into their corresponding formats before being integrated with the encoded categorical features. This process was essential for splitting the dataset into training, evaluation, and test sets. For each set, the features were normalized and integrated with FAISS, and similarity vectors were calculated using a specific k-nearest neighbor approach, yielding distance and indices. Edges were then created between nodes whose similarity distance exceeded a predefined threshold. The similarity graphs were constructed using the normalized features, classification labels, and edges formed previously. Both undirected and directed graphs were generated at this stage. The resulting graphs were saved locally in their respective train, evaluation, and test sets.

The generated graphs were fed into GNN models to begin the training process, where the model parameters were iteratively optimized using graph-structured data. During each epoch, the model produced predictions that were compared to the ground truth labels using the cross-entropy loss function to compute gradients. These gradients were then used to update the model parameters through the Adam optimizer, selected for its adaptive learning rate and proven reliability in deep learning tasks. The model's performance was continually monitored on a validation set, and training continued until the validation error

reached a minimum. This approach helped mitigate overfitting and enhanced the model's generalization to unseen data.

All fixed hyperparameter settings were adopted from the study of Altaf et al. (2023) [47], specifically, the number of hidden channels is set to 32, and the learning rate is configured as 0.01. For each dataset and classification type, experiments were conducted using different graph types (Directed or Undirected), k-values, and GNN algorithms (GCN, GAT, GraphSAGE), with each experiment repeated at least five times. The exploration of hyperparameter impact followed the methodology of Manoharan et al. [48].

4.3. Evaluation Metrics

For the multi-class classification task, the macro-average method plays a crucial role in assessing the effectiveness of an algorithm [49], and was therefore adopted in this study. Given the imbalanced nature of both the NF-BoT-IoT and NF-ToN-IoT datasets, F1-Score was selected as the primary evaluation metric. This metric is particularly important for evaluating model performance. It has also been widely used in numerous past studies to assess model performance in both binary and multi-class classification tasks [36,50].

4.4. Comparison with Other Works

Table 2 presents a comparison of the study's final results with other ML and DL techniques from previous research [43,44,51]. The table highlights that our approach surpassed all other methods in terms of overall performance, as indicated in bold.

Table 2. Comparison of F1-score results between proposed algorithm and other ML and DL algorithms.

Dataset	NF-BoT-IoT	NF-BoT-IoT	NF-ToN-IoT	NF-ToN-IoT
Classification task	Binary	Multi-class	Binary	Multi-class
Proposed approach	0.9852	0.8404	1.0000	0.6325
Decision Tree	0.9848	0.4330	0.9219	0.5460
Random Forest	0.9843	0.8091	0.9224	0.5596
Naïve Bayes	0.9600	0.7654	0.7264	0.2095
E-GraphSAGE	0.9700	0.8100	1.0000	0.6300
Self-supervised Learning	0.9767	0.8270	N/A	N/A
Graph Isomorphism Network	0.9679	0.8211	0.7417	0.6240

4.5. Ablation Study

Table 3 presents the results of an ablation study investigating the impact of different similarity metrics—namely, cosine similarity and Euclidean distance—and the role of the similarity threshold in determining the F1-score performance. Cosine similarity evaluates the angle between feature vectors to assess their orientation similarity, while Euclidean distance measures the absolute magnitude of difference between vectors. Given the high dimensional nature of the IoT dataset used in this study, cosine similarity was expected to outperform Euclidean distance, which is known to be sensitive to scale and less robust in high-dimensional spaces.

In line with the findings of Elsharkawi et al. (2025) [52], the similarity threshold was identified as a critical parameter for enhancing classification performance by controlling the formation of edges between nodes. To mitigate the inclusion of noisy or irrelevant connections while retaining informative ones, we combined the use of a Top-K neighbor selection strategy with a similarity threshold criterion. Specifically, FAISS was employed to efficiently retrieve the Top-K most similar neighbors for each node. We retained this

mechanism throughout the ablation studies due to its computational efficiency, as replacing it with a brute-force full similarity matrix computation would have significantly degraded performance.

The experimental results confirm that cosine similarity consistently outperformed Euclidean distance in this context. Furthermore, the inclusion of a similarity threshold proved effective in improving the quality of the constructed graph by preserving only semantically meaningful edges. Based on the outcomes of ablation studies 3 and 4 in Table 3, along with common practice, a similarity threshold value of 0.8 (referred to as Sim08) was selected for the proposed approach.

Table 3. Ablation study comparing the impact of cosine similarity, Euclidean distance, and similarity thresholds on the average F1-score performance across approximately five trial runs.

Dataset		NF-BoT-IoT	NF-BoT-IoT	NF-ToN-IoT	NF-ToN-IoT
Classification Task		Binary	Multi-Class	Binary	Multi-Class
Proposed approach	Cosine + Sim08 + TopK	0.9852	0.8404	1.0000	0.6325
Ablation study 1	Euclidean + Sim08 + TopK	0.9849	0.8204	1.0000	0.6066
Ablation study 2	Cosine + TopK	0.9852	0.8344	1.0000	0.6265
Ablation study 3	Cosine + Sim06 + TopK	0.9849	0.8375	1.0000	0.6280
Ablation study 4	Cosine + Sim09 + TopK	0.9851	0.8309	1.0000	0.6256

4.6. Binary Classification Results

Figures 2 and 3 present a performance comparison of binary classification results for the NF-BoT-IoT and NF-ToN-IoT datasets, respectively. It is evident that GraphSAGE consistently outperformed the other GNN algorithms. Specifically, GraphSAGE achieved the highest binary classification results, with an F1-score of 0.985222 ($D_{k7_GraphSAGE}$) for the NF-BoT-IoT dataset and 0.999998 ($U_{k7_GraphSAGE}$) for the NF-ToN-IoT dataset, compared to 0.984507 (U_{k7_GCN}) for NF-BoT-IoT using the GCN algorithm, and 0.999994 (U_{k7_GAT}) for NF-ToN-IoT using the GAT algorithm. The binary classification results for the NF-BoT-IoT dataset (Table 4) show that the GAT was the most stable algorithm, as indicated by its small standard deviation. Table 5 demonstrates the consistency in achieving excellent F1-scores across all tested scenarios. For both datasets, binary classification results for the GAT model are not provided for $k = 9$ and $k = 10$, as the processing time for these cases was significantly longer than for the others.

Table 4. Binary classification results of NF-BoT-IoT dataset.

Graph Type	k	GNN Model	Aver F1_Score	Std_Deviation	Max F1_Score	Aver Run Time (sec)	No. Runs
Directed	3	GCN	0.980604	0.008393	0.984425	11.56	5
		GAT	0.965590	0.000000	0.965590	2.00	5
		GraphSAGE	0.985099	0.000075	0.985157	5.61	6
	5	GCN	0.976781	0.010216	0.984286	7.74	5
		GAT	0.965590	0.000000	0.965590	11.86	5
		GraphSAGE	0.985143	0.000083	0.985226	6.38	6
	7	GCN	0.976847	0.010276	0.984354	11.43	5
		GAT	0.965590	0.000000	0.965590	12.29	5
		GraphSAGE	0.985196	0.000071	0.985257	6.67	6

Table 4. Cont.

Graph Type	k	GNN Model	Aver F1_Score	Std_Deviation	Max F1_Score	Aver Run Time (sec)	No. Runs
Directed	9	GCN	0.976807	0.010240	0.984320	10.90	5
		GraphSAGE	0.985222	0.000048	0.985298	8.58	6
	10	GCN	0.980531	0.008352	0.984277	14.96	5
		GraphSAGE	0.983047	0.005238	0.985246	6.23	6
Undirected	3	GCN	0.984411	0.000064	0.984488	10.53	5
		GAT	0.969334	0.008372	0.984310	1.96	5
		GraphSAGE	0.984873	0.000281	0.985262	8.39	6
	5	GCN	0.980686	0.008448	0.984537	8.33	5
		GAT	0.965590	0.000000	0.965590	1.48	5
		GraphSAGE	0.984964	0.000154	0.985094	6.90	6
	7	GCN	0.984507	0.000113	0.984635	10.62	5
		GAT	0.968815	0.007595	0.984310	1.84	6
		GraphSAGE	0.981801	0.007942	0.985110	6.02	6
	9	GCN	0.984504	0.000115	0.984629	11.15	5
		GraphSAGE	0.985096	0.000028	0.985127	8.02	6
	10	GCN	0.984407	0.000062	0.984500	10.95	5
		GraphSAGE	0.985083	0.000077	0.985174	8.06	6

Table 5. Binary classification results of NF-ToN-IoT dataset.

Graph Type	k	GNN Model	Aver F1_Score	Std_Deviation	Max F1_Score	Aver Run Time (sec)	No. Runs
Directed	3	GCN	0.999993	0.000001	0.999996	44.30	6
		GAT	0.999993	0.000000	0.999993	322.21	5
		GraphSAGE	0.999987	0.000005	0.999993	30.37	8
	5	GCN	0.999989	0.000002	0.999993	55.02	6
		GAT	0.999991	0.000003	0.999993	364.50	5
		GraphSAGE	0.999995	0.000003	0.999996	46.29	8
	7	GCN	0.999989	0.000000	0.999989	75.32	6
		GAT	0.999993	0.000004	0.999996	1947.74	6
		GraphSAGE	0.999997	0.000004	1.000000	43.75	6
	9	GCN	0.999981	0.000002	0.999982	103.55	6
		GraphSAGE	0.999992	0.000004	0.999996	37.50	6
	10	GCN	0.999981	0.000005	0.999989	93.41	5
		GraphSAGE	0.999992	0.000005	0.999996	35.64	6
Undirected	3	GCN	0.999988	0.000005	0.999993	36.65	5
		GAT	0.999989	0.000003	0.999993	48.70	5
		GraphSAGE	0.999996	0.000005	1.000000	42.21	6
	5	GCN	0.999990	0.000003	0.999993	32.56	5
		GAT	0.999991	0.000002	0.999993	48.99	5
		GraphSAGE	0.999988	0.000005	0.999993	42.76	7

Table 5. Cont.

Graph Type	k	GNN Model	Aver F1_Score	Std_Deviation	Max F1_Score	Aver Run Time (sec)	No. Runs
Undirected	7	GCN	0.999991	0.000004	0.999996	28.88	5
		GAT	0.999994	0.000002	0.999996	74.74	5
		GraphSAGE	0.999998	0.000002	1.000000	30.73	6
	9	GCN	0.999986	0.000004	0.999993	24.03	6
		GraphSAGE	0.999990	0.000007	1.000000	36.27	6
	10	GCN	0.999984	0.000005	0.999993	24.36	5
		GraphSAGE	0.999988	0.000005	0.999996	25.58	6

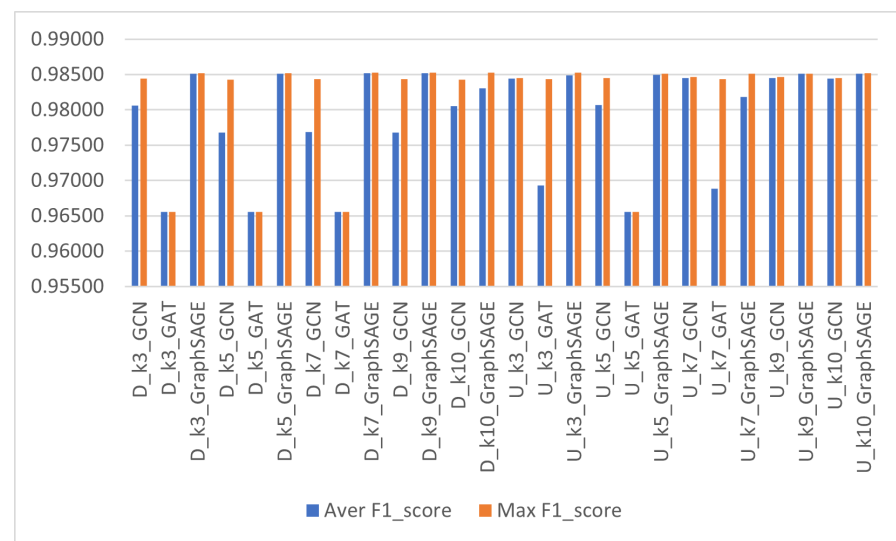


Figure 2. Binary classification performance results on the NF-BoT-IoT dataset using different GNN configurations. The plot compares directed and undirected graph construction types, values of k for Top-k neighbor selection, along with GNN models such as GCN, GAT, and GraphSAGE. For each configuration, both the average and maximum F1-scores are reported.

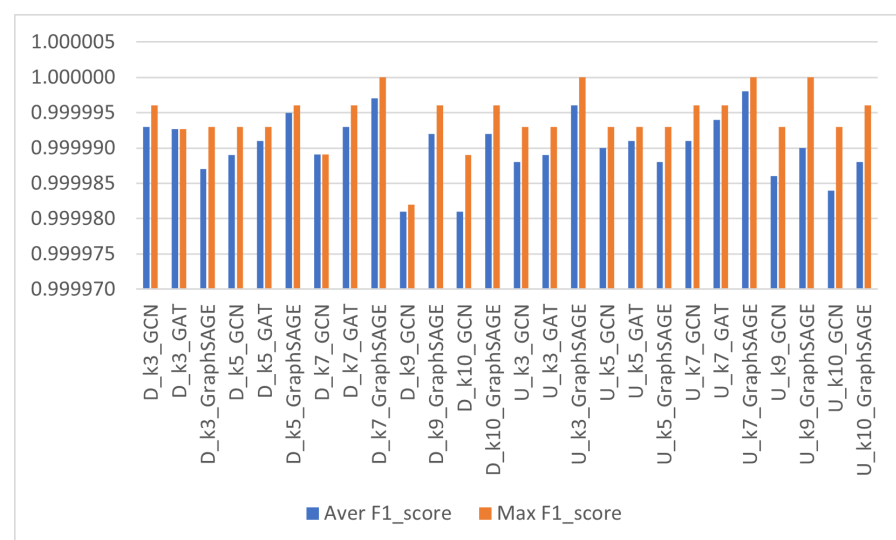


Figure 3. Binary classification performance results on the NF-ToN-IoT dataset using different GNN configurations. The plot compares directed and undirected graph construction types, values of k for Top-k neighbor selection, along with GNN models such as GCN, GAT, and GraphSAGE. For each configuration, both the average and maximum F1-scores are reported.

4.7. Multi-Class Classification Results

Figures 4 and 5 summarize the performance comparison of multi-class classification results for the NF-BoT-IoT and NF-ToN-IoT datasets, respectively. GraphSAGE once again consistently outperformed the other two GNN algorithms. Specifically, GraphSAGE achieved the best multi-class classification results with an F1-score of 0.840447 ($U_{k10_GraphSAGE}$) for the NF-BoT-IoT dataset and 0.628374 ($U_{k5_GraphSAGE}$) for the NF-ToN-IoT dataset, compared to 0.835111 (U_{k3_GCN}) and 0.619946 (U_{k7_GCN}) for the NF-BoT-IoT and NF-ToN-IoT datasets, respectively, using the GCN algorithm. Table 6 highlights the notable performance drop for GAT compared to GCN and GraphSAGE. Although the reduction in GAT performance shown in Table 7 is smaller than in Table 6, the results from both tables underscore the limitations of the GAT algorithm when handling large dynamic graphs. This aligns with the theoretical foundation of the GAT model, which struggles to scale effectively with large graphs. As a result, the multi-class classification results for the GAT model with $k = 3$, $k = 5$, and $k = 7$ settings are only shown for representative purposes.

Table 6. Multi-class classification results for NF-BoT-IoT dataset.

Graph Type	k	GNN Model	Aver F1_Score	Std_Deviation	Max F1_Score	Aver Run Time (sec)	No. Runs
Directed	3	GCN	0.814859	0.009034	0.826520	16.98	5
		GAT	0.749033	0.028207	0.797119	6.15	5
		GraphSAGE	0.834453	0.007781	0.841952	6.48	5
	5	GCN	0.797705	0.056191	0.828939	16.59	5
		GAT	0.725212	0.028222	0.766572	85.13	5
		GraphSAGE	0.834693	0.005104	0.841671	6.80	5
	7	GCN	0.817505	0.016056	0.842142	19.50	5
		GAT	0.747354	0.027544	0.790514	41.76	5
		GraphSAGE	0.838391	0.008680	0.844298	7.54	6
	9	GCN	0.811300	0.011066	0.827032	20.45	5
		GraphSAGE	0.840263	0.009466	0.846004	8.12	6
	10	GCN	0.815999	0.011775	0.828960	25.41	5
		GraphSAGE	0.834799	0.007305	0.845157	9.17	6
Undirected	3	GCN	0.835111	0.011149	0.844220	10.38	5
		GAT	0.732416	0.005847	0.742869	1.06	5
		GraphSAGE	0.822018	0.032488	0.842849	6.03	6
	5	GCN	0.832770	0.008551	0.842253	10.24	5
		GAT	0.729812	0.027002	0.768543	3.34	5
		GraphSAGE	0.832545	0.009027	0.843114	9.30	6
	7	GCN	0.834307	0.007548	0.841951	9.27	5
		GAT	0.739241	0.009096	0.748806	2.61	5
		GraphSAGE	0.814455	0.039932	0.842851	8.48	6
	9	GCN	0.830168	0.007789	0.840973	10.80	5
		GraphSAGE	0.833313	0.010022	0.844539	9.65	6
	10	GCN	0.834366	0.009582	0.843074	10.99	5
		GraphSAGE	0.840447	0.002994	0.843658	9.09	6

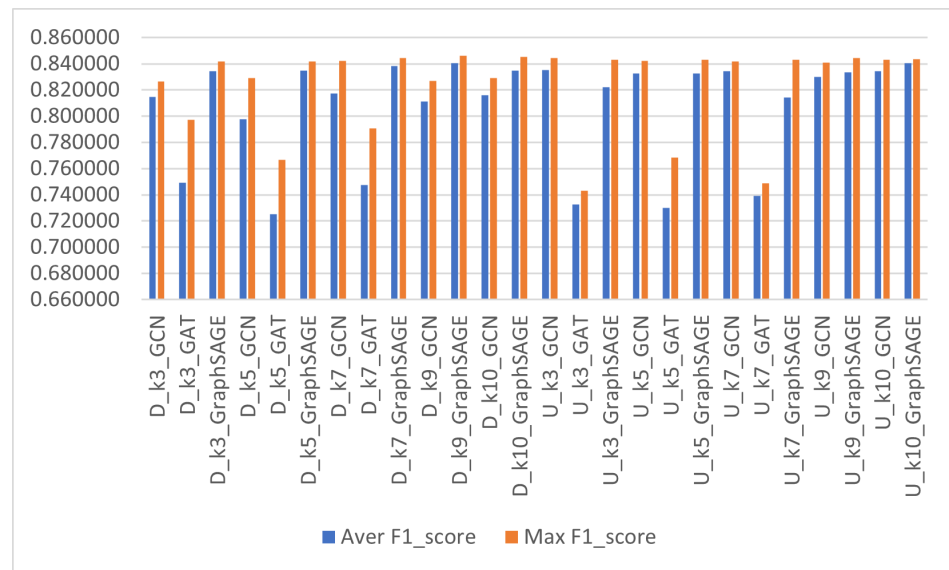


Figure 4. Multi-class classification performance results on the NF-BoT-IoT dataset using different GNN configurations. The plot compares directed and undirected graph construction types, values of k for Top-k neighbor selection along with GNN models such as GCN, GAT, and GraphSAGE. For each configuration, both the average and maximum F1-scores are reported.

Table 7. Multi-class classification results for NF-ToN-IoT dataset.

Graph Type	k	GNN Model	Aver F1_Score	Std_Deviation	Max F1_Score	Aver Run Time (sec)	No. Runs
Directed	3	GCN	0.612570	0.007504	0.624629	25.46	5
		GAT	0.607664	0.004639	0.611103	530.38	5
		GraphSAGE	0.626702	0.004311	0.628980	1029.55	6
	5	GCN	0.616292	0.006024	0.622707	38.52	5
		GAT	0.607655	0.004653	0.611122	960.29	5
		GraphSAGE	0.627931	0.001848	0.629155	1753.62	6
	7	GCN	0.615234	0.007517	0.623258	33.51	6
		GAT	0.608106	0.006721	0.618768	1114.47	6
		GraphSAGE	0.628322	0.000808	0.629565	546.65	6
	9	GCN	0.611154	0.005390	0.620563	41.06	5
		GraphSAGE	0.627101	0.003331	0.629233	65.98	7
	10	GCN	0.612157	0.004555	0.617085	39.49	5
		GraphSAGE	0.627982	0.001686	0.629250	69.14	6
Undirected	3	GCN	0.615036	0.006942	0.622537	25.10	5
		GAT	0.607849	0.008083	0.620632	291.95	5
		GraphSAGE	0.624463	0.004015	0.628668	325.65	8
	5	GCN	0.614611	0.007201	0.621941	27.63	5
		GAT	0.610878	0.005752	0.618631	430.33	5
		GraphSAGE	0.628374	0.000500	0.628892	703.93	5
	7	GCN	0.619946	0.001882	0.622298	32.48	5
		GAT	0.607477	0.004956	0.611158	490.89	5
		GraphSAGE	0.627224	0.002342	0.628887	1189.82	5

Table 7. Cont.

Graph Type	k	GNN Model	Aver F1_Score	Std_Deviation	Max F1_Score	Aver Run Time (sec)	No. Runs
Undirected	9	GCN	0.612081	0.007232	0.620648	29.72	5
		GraphSAGE	0.626142	0.006277	0.629182	47.16	7
	10	GCN	0.617318	0.005695	0.622252	32.57	5
		GraphSAGE	0.627393	0.001871	0.628997	56.05	6

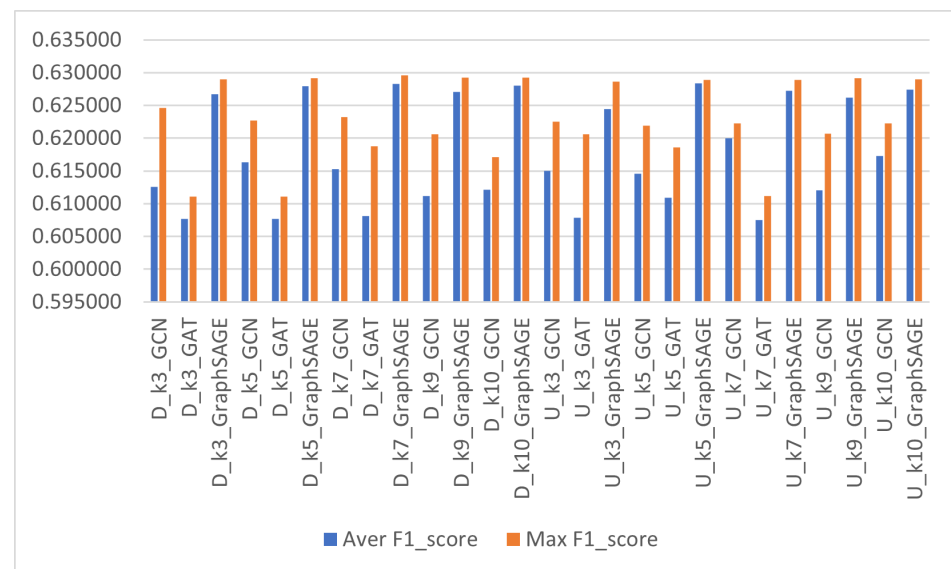


Figure 5. Multi-class classification performance results on the NF-ToN-IoT dataset using different GNN configurations. The plot compares directed and undirected graph construction types, values of k for Top- k neighbor selection, along with GNN models such as GCN, GAT, and GraphSAGE. For each configuration, both the average and maximum F1-scores are reported.

4.8. GNN Model Structure Exploration

Building on the findings from the preceding sections, which emphasized the impact of graph type and structural parameters on classification performance and consequently inform model selection and architectural design for IoT cyber threat detection, this section provides an in-depth exploration of the structural characteristics and performance implications of the different GNN models.

For each dataset, we followed the approach of Halbouni et al. (2022) to investigate the impact of batch normalization, dropout, and residual connections on the F1-score performance of each tested GNN model, in both binary and multi-class classifications [36]. This exploration aimed to identify the best-performing variant of the GraphSAGE models, providing valuable insights for future studies to understand the effects of these parameters. Table 8 presents a list of the different GraphSAGE variants, along with the sequence in which their functions were applied.

Table 9 presents the top three GraphSAGE variants for each dataset, using the same hyperparameters that yielded the best results for the corresponding classification tasks. The results show that adding a residual connection and an additional batch normalization layer can improve overall performance, as demonstrated by the GraphSAGE3 variant. It was also observed that smaller datasets may benefit from using a larger k -value ($k = 9$ and $k = 10$) compared to $k = 7$ and $k = 5$ in order to optimize the F1-score performance. Another key observation from Table 9 is that undirected graphs tended to perform better on multi-class classification tasks, while directed graphs are more effective for binary

classification tasks. However, these findings should be further validated with additional datasets for more conclusive results.

Table 8. List of encoded GraphSAGE variants and the order of their applied functions.

GraphSAGE Variant	Applied Function					
	1st Layer Dropout	1st Layer BatchNorm	Activation Function	Residual	2nd Layer Dropout	2nd Layer BatchNorm
GraphSAGE	N/A	1st	2nd	N/A	3rd	N/A
GraphSAGE1	N/A	1st	2nd	N/A	3rd	4th
GraphSAGE2	N/A	1st	3rd	2nd	4th	5th
GraphSAGE3	N/A	1st	2nd	3rd	4th	5th
GraphSAGE4	N/A	2nd	3rd	1st	4th	5th
GraphSAGE5	N/A	2nd	1st	N/A	3rd	4th
GraphSAGE6	N/A	2nd	1st	3rd	4th	5th
GraphSAGE7	N/A	3rd	1st	2nd	4th	5th
GraphSAGE8	N/A	3rd	2nd	1st	4th	5th
GraphSAGE9	N/A	N/A	1st	N/A	2nd	N/A
GraphSAGE10	1st	N/A	2nd	N/A	3rd	N/A

Table 9. Ranking of best GraphSAGE variants for different classification tasks.

Dataset	Classify Type	Graph Type	k	Ranking	GraphSAGE Variant	Aver F1_Score	Std_Deviation	Max F1_Score	Aver Run Time (sec)	No. Runs
NF-BoT-IoT	Binary	Directed	9	1	GraphSAGE	0.985222	0.000048	0.985298	8.58	6
				2	GraphSAGE9	0.985031	0.000212	0.985262	28.56	5
				3	GraphSAGE5	0.974149	0.010756	0.985196	56.01	5
NF-BoT-IoT	Multi	Undirected	10	1	GraphSAGE	0.840447	0.002994	0.843658	9.09	6
				2	GraphSAGE9	0.835847	0.006773	0.845202	33.69	5
				3	GraphSAGE1	0.822281	0.018911	0.843293	48.28	5
NF-ToN-IoT	Binary	Directed	7	1	GraphSAGE9	0.999999	0.000002	1.000000	123.69	5
				2	GraphSAGE	0.999997	0.000004	1.000000	43.75	6
				3	GraphSAGE7	0.999992	0.000007	1.000000	990.92	5
NF-ToN-IoT	Multi	Undirected	5	1	GraphSAGE3	0.632464	0.013479	0.656238	929.90	5
				2	GraphSAGE6	0.628516	0.000215	0.628872	997.19	5
				3	GraphSAGE	0.628374	0.000500	0.628892	703.93	5

Since there is limited insight into how well the model classified specific types of attacks, we chose to present a multi-class classification confusion matrix comparison between datasets to highlight these details. Figures 6 and 7 show the confusion matrices for the NF-BoT IoT and NF-ToN IoT datasets, respectively, illustrating the performance of the GraphSAGE algorithm in classifying different attack types.

For the NF-BoT IoT dataset, the confusion matrix indicates that the GraphSAGE algorithm performed well in detecting Benign (0.45), DDoS (0.49), DoS (0.46), and Reconnaissance (0.94) attacks, but struggled with identifying Theft (0.01) attacks. Notably, there were misclassifications between Benign and Reconnaissance (0.53), as well as Theft and Reconnaissance (0.90). On the NF-ToN IoT dataset, the algorithm demonstrated high accuracy in classifying Benign (1.00), Backdoor (0.92), DDoS (0.70), and Injection (0.90) attacks. However, it performed poorly in detecting DoS (0), MITM (0), Ransomware (0), Scanning (0), and XSS (0) attacks, with significant confusion between these attack types

and others like DoS and DDoS. Specifically, all DoS attacks were classified as DDoS. This issue arose because GraphSAGE does not incorporate edge features when computing new embeddings, making it unable to distinguish between these types of attacks. This highlights an opportunity to explore the application of our proposed framework within the E-GraphSAGE model in future studies.

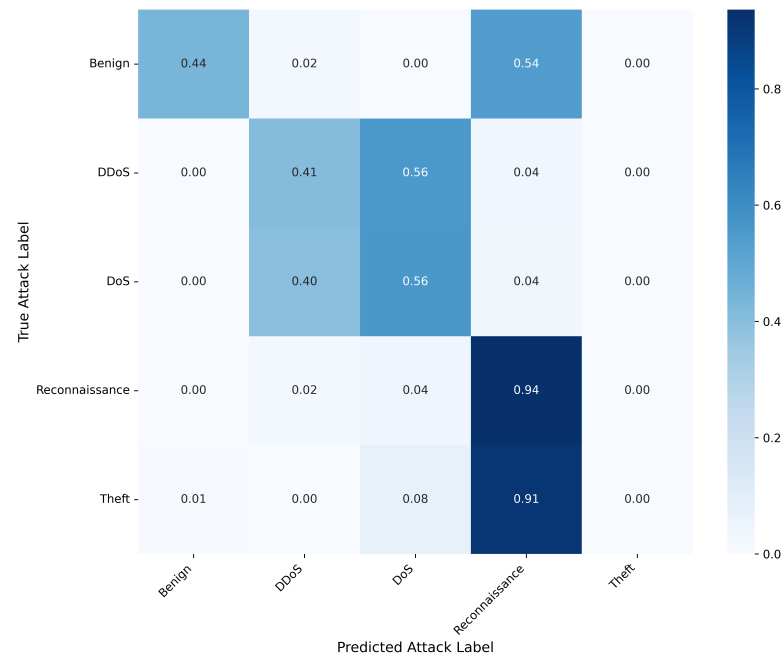


Figure 6. Confusion matrix for multi-class classification on the NF-BoT-IoT dataset using the GraphSAGE model with an undirected graph structure and $k = 10$.

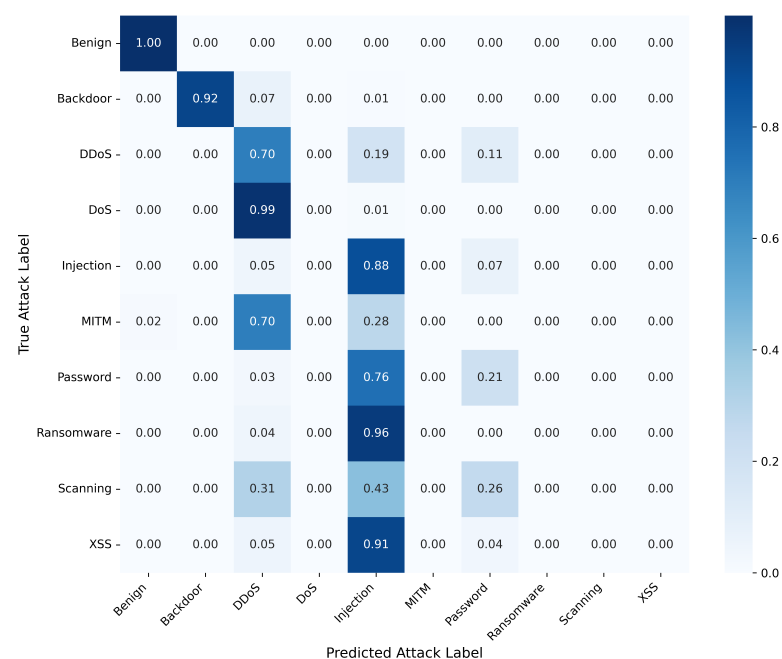


Figure 7. Confusion matrix for multi-class classification on the NF-ToN-IoT dataset using the GraphSAGE3 model with an undirected graph structure and $k = 5$.

5. Conclusions

This paper introduced a Top-K Similarity Graph Framework for IoT cyber threat detection, aiming to provide a more meaningful representation of node relationships.

Our proposed framework demonstrated superior performance in both binary and multi-class classification tasks using the NF-ToN IoT and NF-BoT IoT datasets, outperforming traditional machine learning methods and existing graph-based approaches. We also explored the effects of graph directionality (directed vs. undirected), varying K values, and different GNN architectures and configurations on IoT NIDS detection performance. The results showed that GraphSAGE consistently yielded the best performance, and further hyperparameter tuning, coupled with batch normalization, dropout, and residual connections, could boost the average F1-Scores to 98.52% and 84.04% for NF-BoT-IoT binary and multi-class classification, as well as nearly 100% and 63.25% for NF-ToN-IoT binary and multi-class classification. However, the framework faced challenges in detecting specific attacks, such as Theft, DOS, MITM, Password, Ransomware, Scanning, and XSS. Despite these limitations, the experiments provided valuable insights and can serve as a foundation for future research and practical applications in the field. Future work could integrate the proposed framework with the E-GraphSAGE model to enhance detection rates for these attacks. The implementation is publicly available at <https://github.com/tngo88/TKSGF> (accessed on 8 June 2025).

Author Contributions: Conceptualization, J.Y.; Data curation, T.N.; Formal analysis, Y.-F.G.; Investigation, T.N.; Methodology, T.N. and J.Y.; Resources, H.W.; Supervision, J.Y., Y.-F.G. and H.W.; Validation, T.N.; Visualization, T.N.; Writing—original draft, T.N.; Writing—review and editing, J.Y., Y.-F.G. and H.W. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Data is contained within the article.

Conflicts of Interest: The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Rejeb, A.; Rejeb, K.; Treiblmaier, H.; Appolloni, A.; Alghamdi, S.; Alhasawi, Y.; Iranmanesh, M. The Internet of Things (IoT) in healthcare: Taking stock and moving forward. *Internet Things* **2023**, *22*, 100721. [\[CrossRef\]](#)
2. Sinha, B.B.; Dhanalakshmi, R. Recent advancements and challenges of Internet of Things in smart agriculture: A survey. *Future Gener. Comput. Syst.* **2022**, *126*, 169–184. [\[CrossRef\]](#)
3. Singh, P.; Elmi, Z.; Meriga, V.K.; Pasha, J.; Dulebenets, M.A. Internet of Things for sustainable railway transportation: Past, present, and future. *Clean. Logist. Supply Chain* **2022**, *4*, 100065. [\[CrossRef\]](#)
4. Li, Y.; Liu, Q. A comprehensive review study of cyber-attacks and cyber security; Emerging trends and recent developments. *Energy Rep.* **2021**, *7*, 8176–8186. [\[CrossRef\]](#)
5. Manoharan, P.; Hong, W.; Yin, J.; Wang, H.; Zhang, Y.; Ye, W. Optimising Insider Threat Prediction: Exploring BiLSTM Networks and Sequential Features. In *Data Science and Engineering*; Springer Nature: Cham, Switzerland, 2024; pp. 1–16.
6. Yin, J.; Tang, M.; Cao, J.; You, M.; Wang, H. Cybersecurity applications in software: Data-driven software vulnerability assessment and management. In *Emerging Trends in Cybersecurity Applications*; Springer: Cham, Switzerland, 2022; pp. 371–389.
7. Guerra, J.L.; Catania, C.; Veas, E. Datasets are not enough: Challenges in labeling network traffic. *Comput. Secur.* **2022**, *120*, 102810. [\[CrossRef\]](#)
8. Eid, A.M.; Soudan, B.; Nassif, A.B.; Injadat, M. Comparative study of ML models for IIoT intrusion detection: Impact of data preprocessing and balancing. *Neural Comput. Appl.* **2024**, *36*, 6955–6972. [\[CrossRef\]](#)
9. Mbow, M.; Koide, H.; Sakurai, K. An intrusion detection system for imbalanced dataset based on deep learning. In Proceedings of the 2021 Ninth International Symposium on Computing and Networking (CANDAR), Matsue, Japan, 23–26 November 2021; pp. 38–47.
10. Wang, Y.; Li, D.; Li, X.; Yang, M. PC-GAIN: Pseudo-label conditional generative adversarial imputation networks for incomplete data. *Neural Netw.* **2021**, *141*, 395–403. [\[CrossRef\]](#) [\[PubMed\]](#)
11. Chen, Y.; Zhang, Y.; Maharjan, S. Deep learning for secure mobile edge computing. *arXiv* **2017**, arXiv:1709.08025.

12. Abdulganiyu, O.H.; Ait Tchakoucht, T.; Saheed, Y.K. A systematic literature review for network intrusion detection system (IDS). *Int. J. Inf. Secur.* **2023**, *22*, 1125–1162. [\[CrossRef\]](#)
13. Yu, Y.; Zeng, X.; Xue, X.; Ma, J. LSTM-based intrusion detection system for VANETs: A time series classification approach to false message detection. *IEEE Trans. Intell. Transp. Syst.* **2022**, *23*, 23906–23918. [\[CrossRef\]](#)
14. Wu, Z.; Zhang, H.; Wang, P.; Sun, Z. RTIDS: A robust transformer-based approach for intrusion detection system. *IEEE Access* **2022**, *10*, 64375–64387. [\[CrossRef\]](#)
15. Kheddar, H. Transformers and large language models for efficient intrusion detection systems: A comprehensive survey. *arXiv* **2024**, arXiv:2408.07583. [\[CrossRef\]](#)
16. Aldwairi, M.; Jarrah, M.; Mahasneh, N.; Al-khateeb, B. Graph-based data management system for efficient information storage, retrieval and processing. *Inf. Process. Manag.* **2023**, *60*, 103165. [\[CrossRef\]](#)
17. Liu, M.; Li, X.; Li, J.; Liu, Y.; Zhou, B.; Bao, J. A knowledge graph-based data representation approach for IIoT-enabled cognitive manufacturing. *Adv. Eng. Inform.* **2022**, *51*, 101515. [\[CrossRef\]](#)
18. Bilot, T.; El Madhoun, N.; Al Agha, K.; Zouaoui, A. Graph neural networks for intrusion detection: A survey. *IEEE Access* **2023**, *11*, 49114–49139. [\[CrossRef\]](#)
19. Juan, X.; Zhou, F.; Wang, W.; Jin, W.; Tang, J.; Wang, X. INS-GNN: Improving graph imbalance learning with self-supervision. *Inf. Sci.* **2023**, *637*, 118935. [\[CrossRef\]](#)
20. Wu, C.; Wu, F.; Cao, Y.; Huang, Y.; Xie, X. Fedgnn: Federated graph neural network for privacy-preserving recommendation. *arXiv* **2021**, arXiv:2102.04925.
21. Xu, K.; Li, Y.; Li, Y.; Xu, L.; Li, R.; Dong, Z. Masked graph neural networks for unsupervised anomaly detection in multivariate time series. *Sensors* **2023**, *23*, 7552. [\[CrossRef\]](#)
22. Yin, J.; Hong, W.; Wang, H.; Cao, J.; Miao, Y.; Zhang, Y. A compact vulnerability knowledge graph for risk assessment. *ACM Trans. Knowl. Discov. Data* **2024**, *18*, 1–17. [\[CrossRef\]](#)
23. Wang, Z.; Eisen, M.; Ribeiro, A. Learning decentralized wireless resource allocations with graph neural networks. *IEEE Trans. Signal Process.* **2022**, *70*, 1850–1863. [\[CrossRef\]](#)
24. Munikoti, S.; Agarwal, D.; Das, L.; Halappanavar, M.; Natarajan, B. Challenges and opportunities in deep reinforcement learning with graph neural networks: A comprehensive review of algorithms and applications. *IEEE Trans. Neural Netw. Learn. Syst.* **2023**, *35*, 15051–15071. [\[CrossRef\]](#)
25. Yin, J.; Chen, G.; Hong, W.; Cao, J.; Wang, H.; Miao, Y. A heterogeneous graph-based semi-supervised learning framework for access control decision-making. *World Wide Web* **2024**, *27*, 35. [\[CrossRef\]](#)
26. Vrahatis, A.G.; Lazaros, K.; Kotsiantis, S. Graph attention networks: A comprehensive review of methods and applications. *Future Internet* **2024**, *16*, 318. [\[CrossRef\]](#)
27. Hajibabae, P.; Malekzadeh, M.; Heidari, M.; Zad, S.; Uzuner, O.; Jones, J.H. An empirical study of the graphsage and word2vec algorithms for graph multiclass classification. In Proceedings of the 2021 IEEE 12th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON), Vancouver, BC, Canada, 27–30 October 2021; pp. 0515–0522.
28. Guezzaz, A.; Benkirane, S.; Azrou, M.; Khurram, S. A reliable network intrusion detection approach using decision tree with enhanced data quality. *Secur. Commun. Netw.* **2021**, *2021*, 1230593. [\[CrossRef\]](#)
29. Majidian, S.Z.; TaghipourEivazi, S.; Arasteh, B.; Ghaffari, A. Optimizing random forests to detect intrusion in the Internet of Things. *Comput. Electr. Eng.* **2024**, *120*, 109860. [\[CrossRef\]](#)
30. Mohy-Eddine, M.; Guezzaz, A.; Benkirane, S.; Azrou, M. An efficient network intrusion detection model for IoT security using K-NN classifier and feature selection. *Multimed. Tools Appl.* **2023**, *82*, 23615–23633. [\[CrossRef\]](#)
31. Mehmood, A.; Mukherjee, M.; Ahmed, S.H.; Song, H.; Malik, K.M. NBC-MAIDS: Naïve Bayesian classification technique in multi-agent system-enriched IDS for securing IoT against DDoS attacks. *J. Supercomput.* **2018**, *74*, 5156–5170. [\[CrossRef\]](#)
32. Alqahtani, M.; Mathkour, H.; Ben Ismail, M.M. IoT botnet attack detection based on optimized extreme gradient boosting and feature selection. *Sensors* **2020**, *20*, 6336. [\[CrossRef\]](#) [\[PubMed\]](#)
33. Douiba, M.; Benkirane, S.; Guezzaz, A.; Azrou, M. An improved anomaly detection model for IoT security using decision tree and gradient boosting. *J. Supercomput.* **2023**, *79*, 3392–3411. [\[CrossRef\]](#)
34. Hamidouche, M.; Popko, E.; Ouni, B. Enhancing iot security via automatic network traffic analysis: The transition from machine learning to deep learning. In Proceedings of the 13th International Conference on the Internet of Things, Nagoya, Japan, 7–10 November 2023; pp. 105–112.
35. Yin, J.; Tang, M.; Cao, J.; Wang, H. Apply transfer learning to cybersecurity: Predicting exploitability of vulnerabilities by description. *Knowl.-Based Syst.* **2020**, *210*, 106529. [\[CrossRef\]](#)
36. Halbouni, A.; Gunawan, T.S.; Habaebi, M.H.; Halbouni, M.; Kartiwi, M.; Ahmad, R. CNN-LSTM: Hybrid deep neural network for network intrusion detection system. *IEEE Access* **2022**, *10*, 99837–99849. [\[CrossRef\]](#)
37. Ullah, I.; Mahmoud, Q.H. Design and development of RNN anomaly detection model for IoT networks. *IEEE Access* **2022**, *10*, 62722–62750. [\[CrossRef\]](#)

38. Park, C.; Lee, J.; Kim, Y.; Park, J.G.; Kim, H.; Hong, D. An enhanced AI-based network intrusion detection system using generative adversarial networks. *IEEE Internet Things J.* **2022**, *10*, 2330–2345. [[CrossRef](#)]
39. Alrayes, F.S.; Zakariah, M.; Amin, S.U.; Khan, Z.I.; Helal, M. Intrusion detection in IoT systems using denoising autoencoder. *IEEE Access* **2024**, *12*, 122401–122425. [[CrossRef](#)]
40. Deng, X.; Zhu, J.; Pei, X.; Zhang, L.; Ling, Z.; Xue, K. Flow topology-based graph convolutional network for intrusion detection in label-limited IoT networks. *IEEE Trans. Netw. Serv. Manag.* **2022**, *20*, 684–696. [[CrossRef](#)]
41. Zhang, L.; Tan, L.; Shi, H.; Sun, H.; Zhang, W. Malicious Traffic Classification for IoT based on Graph Attention Network and Long Short-Term Memory Network. In Proceedings of the 2023 24th Asia-Pacific Network Operations and Management Symposium (APNOMS), Sejong, Republic of Korea, 6–8 September 2023; pp. 54–59.
42. Hamilton, W.; Ying, Z.; Leskovec, J. Inductive representation learning on large graphs. *Adv. Neural Inf. Process. Syst.* **2017**, *30*, 1025–1035.
43. Lo, W.W.; Layeghy, S.; Sarhan, M.; Gallagher, M.; Portmann, M. E-graphsage: A graph neural network based intrusion detection system for iot. In Proceedings of the NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium, Budapest, Hungary, 25–29 April 2022; pp. 1–9.
44. Xu, R.; Wu, G.; Wang, W.; Gao, X.; He, A.; Zhang, Z. Applying self-supervised learning to network intrusion detection for network flows with graph neural network. *Comput. Netw.* **2024**, *248*, 110495. [[CrossRef](#)]
45. Sarhan, M.; Layeghy, S.; Moustafa, N.; Portmann, M. Netflow datasets for machine learning-based network intrusion detection systems. In Proceedings of the Big Data Technologies and Applications: 10th EAI International Conference, BDTA 2020, and 13th EAI International Conference on Wireless Internet, WiCON 2020, Virtual, 11 December 2020; Proceedings 10; Springer: Cham, Switzerland, 2021, pp. 117–135.
46. Mohurle, S.; Patil, M. A brief study of wannacry threat: Ransomware attack 2017. *Int. J. Adv. Res. Comput. Sci.* **2017**, *8*, 1938–1940.
47. Altaf, T.; Wang, X.; Ni, W.; Yu, G.; Liu, R.P.; Braun, R. A new concatenated multigraph neural network for IoT intrusion detection. *Internet Things* **2023**, *22*, 100818. [[CrossRef](#)]
48. Manoharan, P.; Yin, J.; Wang, H.; Zhang, Y.; Ye, W. Insider threat detection using supervised machine learning algorithms. In *Telecommunication Systems*; Springer: Berlin/Heidelberg, Germany, 2023; pp. 1–17.
49. Beaver, J.M.; Borges-Hink, R.C.; Buckner, M.A. An evaluation of machine learning methods to detect malicious SCADA communications. In Proceedings of the 2013 12th International Conference on Machine Learning and Applications, Miami, FL, USA, 4–7 December 2013; Volume 2, pp. 54–59.
50. Ahmad, Z.; Shahid Khan, A.; Wai Shiang, C.; Abdullah, J.; Ahmad, F. Network intrusion detection system: A systematic study of machine learning and deep learning approaches. *Trans. Emerg. Telecommun. Technol.* **2021**, *32*, e4150. [[CrossRef](#)]
51. Ngo, T.; Yin, J.; Ge, Y.F.; Zhou, C.; Cao, J. Comparative Study of Machine Learning Algorithms for IoT Cyber Threat Detection in Healthcare Information Systems. In Proceedings of the International Conference on Health Information Science, Hong Kong, China, 8–10 December 2024; Springer: Singapore, 2025; pp. 68–77.
52. Elsharkawi, I.; Sharara, H.; Rafea, A. SViG: A Similarity-thresholded Approach for Vision Graph Neural Networks. *IEEE Access* **2025**, *13*, 19379–19387. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.